

Competitor Price and Availability Intelligence Platform

An end-to-end data platform that tracks competitor prices across retailers, records every change, forecasts short-term movement, matches equivalent products across catalogues, and alerts when a competitor undercuts us.

SUBMITTED BY Love Vyas	DATE 1 September 2026	DOMAIN Retail price intelligence
CORE STACK Python · Celery · PostgreSQL · dbt	TARGET CLOUD AWS ap-south-1 (Mumbai)	SCALE 8,433 price observations

Contents

1. Problem statement

2. Objectives and scope

3. Solution overview

4. Technology stack and where each part is used

5. Data sources

6. Module 1 — Ingestion

7. Module 2 — Change data capture

8. Module 3 — Database design

9. Module 4 — Transformation
10. Module 5 — Machine learning: forecasting

11. Module 6 — Machine learning: product matching

12. Module 7 — Agentic AI: the weekly brief

13. Module 8 — Presentation layer

14. Field legend (data dictionary)

15. AWS deployment plan

16. Security and access control

17. Testing and data quality

18. Results

19. Roadmap

Section 1 — Problem Statement

Business context

Retailers and brands lose margin in two ways that stay invisible without continuous monitoring. A competitor quietly drops a price and takes volume before anyone notices, or a competitor goes out of stock and the demand that would have gone to them is never captured. Commercial products such as Prisync, Competera and Minderest sell exactly this capability: competitor price monitoring, repricing signals and market intelligence.

The problem

Build a system that continuously monitors competitor prices and availability across multiple retailers, records the full history of every change rather than only the latest value, and turns that history into decisions a category manager can act on: who is undercutting us and by how much, which products are volatile enough to need watching, and where prices are likely to move next.

The system must work from public data sources, must not lose history when a source is temporarily unavailable, and must be explicit about the confidence of what it reports: a price observed three weeks ago should not carry the same authority as one observed today.

Expected outcome

A working pipeline that converts raw competitor price observations into a queryable analytical layer, a set of business marts answering specific pricing questions, short-term per-SKU forecasts with measured accuracy, cross-retailer product matching, an alerting path for undercutting, and reporting surfaces for both technical and business users.

Section 2 — Objectives and Scope

OBJECTIVE	SUCCESS CRITERION	OUTCOME
Multi-source ingestion	Pluggable source interface; adding a source is one file	ACHIEVED
Complete change history	Every price and attribute change recorded, never overwritten	ACHIEVED
Reproducibility	Warehouse rebuildable from stored raw payloads with no API calls	ACHIEVED
Tested transformations	Business rules defined once and covered by automated data tests	ACHIEVED
Forecasting	Per-SKU forecast with backtested error inside a 12% target	ACHIEVED
Entity resolution	Cross-catalogue product matching with human review	ACHIEVED
Undercut alerting	Alerts with severity and evidence age, deduplicated	ACHIEVED
Presentation	Dashboard, API, review screen and written brief	ACHIEVED

In scope: ingestion from public APIs, change data capture, warehouse modelling, forecasting, product matching, alerting, and reporting.

Out of scope: web scraping, checkout or transactions, multi-tenancy, and sub-minute streaming. No personal data is collected; the platform holds public product prices only.

Section 3 — Solution Overview

The platform is organised as four stages and eight modules. Data is acquired on a schedule, stored twice (raw and modelled), transformed into tested business marts, and presented through five surfaces. Every presentation surface reads the marts, so a definition such as "what counts as an undercut" exists in exactly one place.

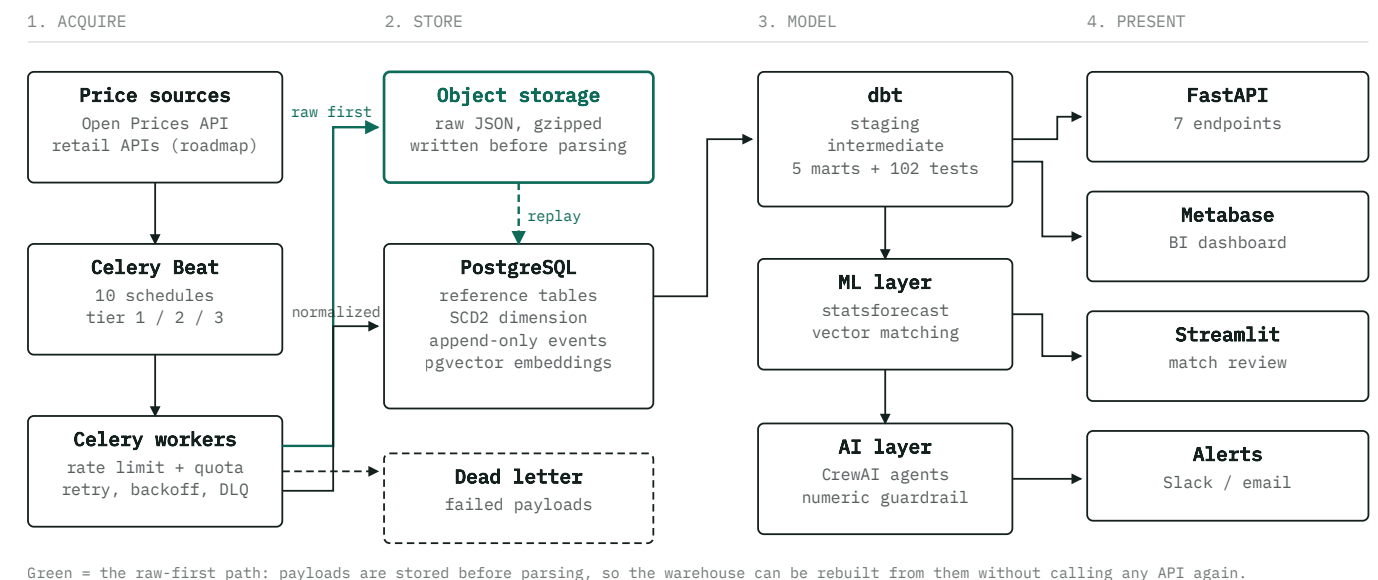


Figure 1. End-to-end architecture. Data moves left to right through four stages; every presentation surface reads from the modelled marts, never from raw tables.

Section 4 — Technology Stack and Where Each Part Is Used

Each technology below is listed with the specific job it does in this system, so its role is unambiguous. Items marked in **green** are the machine learning, vector search and agentic AI components.

TECHNOLOGY	WHERE IT IS USED HERE	PURPOSE
Celery + Celery Beat	Runs all 10 scheduled jobs: tier refreshes, mart rebuild, nightly forecast, matching, alert evaluation, weekly brief, monthly partition creation	Orchestration. Lighter than Airflow for periodic polling with task fan-out
Redis	Two distinct jobs: message broker for Celery, and shared state for the rate limiter (token bucket) and daily API quota counters	Broker + coordination. Shared so limits hold across every worker process
PostgreSQL 16	Serving database: reference tables, SCD2 attribute history, append-only price and stock events, and all derived output	System of record for modelled data
pgvector (vector database)	The <code>product_versions.embedding</code> column stores a 384-dimension vector per product, indexed with HNSW. Used only by product matching, for approximate nearest-neighbour search	Vector search inside PostgreSQL, so no separate vector store is operated
Object storage (S3)	The raw zone: every API payload is written here gzipped before parsing, partitioned by source and date	Durable audit trail and the source for rebuilding the warehouse
dbt Core	Transformation: 6 staging views, 3 intermediate views, 5 marts, and 102 data tests that run with the build	Business logic defined once, in tested SQL
Pydantic v2	Validates each normalized record's shape as it is parsed from a source payload	Per-record contract at the ingestion boundary
Pandera	Validates the whole parsed batch before it reaches the database (catches an all-null column or an empty source)	Cross-record contract that per-record checks cannot see

TECHNOLOGY	WHERE IT IS USED HERE	PURPOSE
Nixtla statsforecast (ML)	Nightly per-SKU price forecasting: AutoETS and AutoARIMA candidates against a SeasonalNaive baseline, scored by rolling-origin cross-validation	Statistical time-series modelling with native prediction intervals
fastembed / all-MiniLM-L6-v2 (ML)	Turns title + brand + category into the 384-dimension vector stored in pgvector	Sentence embeddings via ONNX runtime, avoiding a PyTorch dependency
CrewAI (agentic AI)	Weekly pricing brief: three agents (analyst, forecast interpreter, writer) turn a fixed facts payload into prose	Role-based agent orchestration for the one narrative task
FastAPI	REST API with 7 endpoints, API-key authentication, and generated OpenAPI documentation	Serving layer for programmatic consumers
Metabase	Business dashboard: scorecards, undercut table, volatility leaderboard, price trends, forecast accuracy	Self-service BI connected straight to the marts
Streamlit	Human-in-the-loop screen where a reviewer approves or rejects candidate product matches	Internal review UI, minimal code
Alembic	Versioned schema migrations, including partition creation and index management	Reproducible database evolution
Terraform	Defines the entire AWS target: VPC, subnets, EC2, RDS, S3, SSM, IAM, CloudWatch, budget alarm	Infrastructure as code
Docker Compose	Local runtime for PostgreSQL and Redis; production image for API, worker and scheduler	Consistent environments
structlog	JSON-structured logs from workers and API, ready for CloudWatch ingestion	Machine-readable operational logging
pytest	150 automated tests covering normalizers, CDC rules, forecasting metrics, the numeric guard, alerting and authentication	Automated verification

Section 5 — Data Sources

The platform is built on **Open Prices**, the price project of Open Food Facts: a public API carrying crowd-sourced retail prices, each with a real observation date and an OpenStreetMap store location. That structure matters, because one product barcode genuinely appears at several named retailers over time, which is the exact shape a competitor-price platform needs. Product attributes (name, brand, category) come from the Open Food Facts catalogue embedded in the same payload.

SOURCE	AUTHENTICATION	ROLE IN THE PLATFORM
Open Prices / Open Food Facts	None required	Primary source. All price, retailer and product data.
Fake Store API	None required	Deterministic fixture used by the automated tests.
Best Buy Developer API	API key	Roadmap. Adds stock levels and a US electronics catalogue.
eBay Browse API	OAuth 2.0	Roadmap. Large live catalogue with high price variance.
Digi-Key Product Information	OAuth 2.0	Roadmap. Component pricing with quantity tiers and real-time stock.

The source interface is deliberately narrow: a source implements how to fetch a payload and how to map it to the internal record shape. Ingestion, change data capture, transformation, forecasting and alerting are all source-independent, so adding one of the roadmap sources is a single new file rather than a change to the pipeline.

One authored dataset. Every price, product, retailer and date is real and arrives from the API. The single exception is `own_catalog`, which represents the company's own price list — the thing competitor prices are compared against. No public API can supply that, so it is generated from products actually observed in the last 60 days, with our price set around each observed market average.

Section 6 — Module 1: Ingestion

Ingestion is scheduled, rate-limited and failure-aware. Products are assigned a **tier** that decides how often they are re-checked, which concentrates limited API quota on the products that move most: tier 1 every six hours, tier 2 daily, tier 3 weekly.

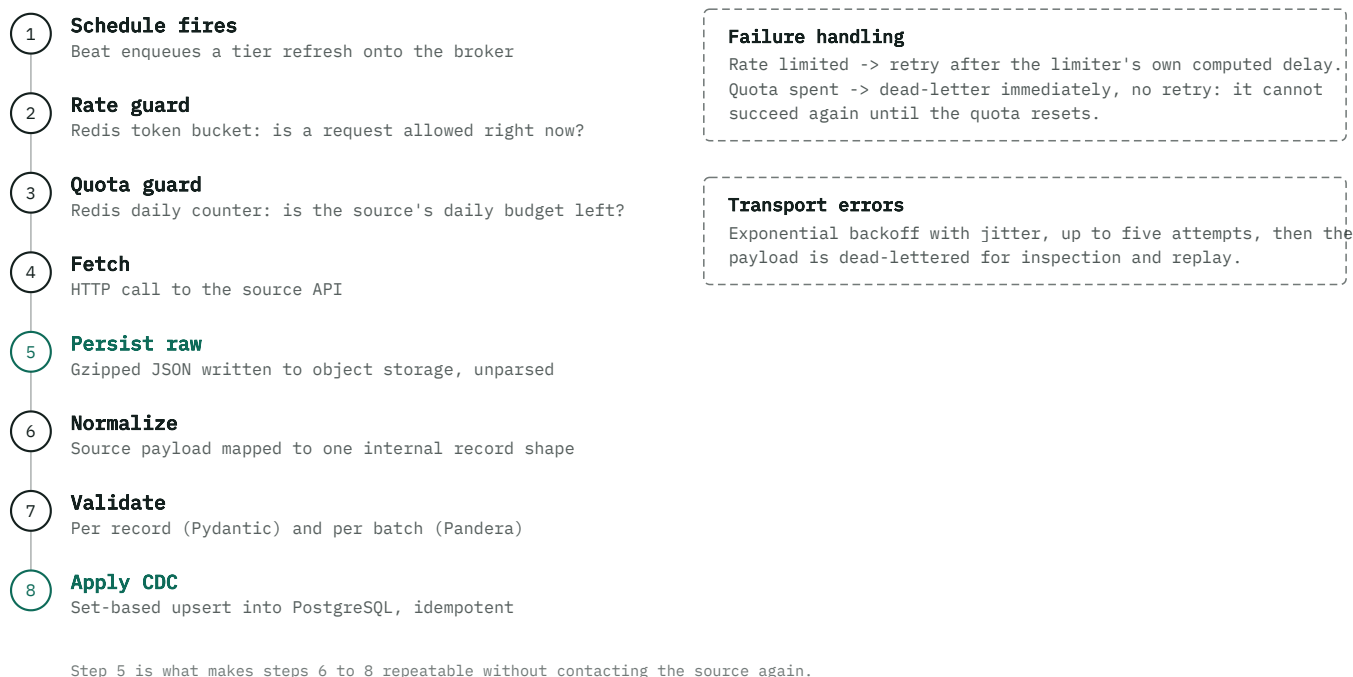


Figure 2. Ingestion module. Two independent Redis guards sit in front of every HTTP call, and the raw payload is durable before any parsing logic runs.

Guarding the source APIs

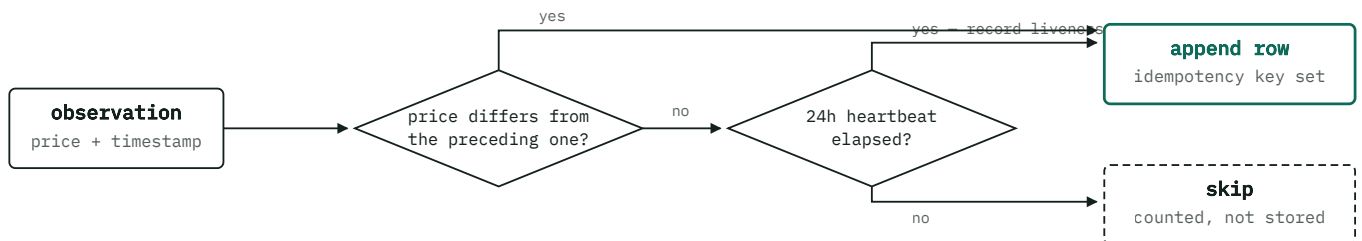
Two independent guards sit in front of every HTTP call, both held in Redis so the limit applies across all worker processes rather than per worker:

- A **token bucket** smooths the instantaneous request rate and allows a controlled burst.
- A **daily counter** enforces a published quota, such as eBay's documented 5,000 calls per day.

The two failure modes are handled differently on purpose. A rate limit is temporary, so the task retries after the delay the limiter itself computes. An exhausted daily quota cannot succeed again today, so the task is dead-lettered immediately rather than consuming retries that are certain to fail.

Section 7 — Module 2: Change Data Capture

The platform separates *what a product is* from *what was observed about it*. Product attributes are held as a Type 2 slowly changing dimension: when a tracked attribute changes, the open row is closed with an end timestamp and a new row is opened. Price observations are appended to an immutable event table. Nothing is ever updated in place, so the price on any past date can always be reconstructed.



The comparison uses the observation immediately PRECEDING this one in time, not the newest row, so backfilled history is judged against the right ne:

Figure 3. Change data capture. Unchanged prices are not re-recorded, which keeps the event tables lean; the heartbeat still records that the system looked and saw no change.

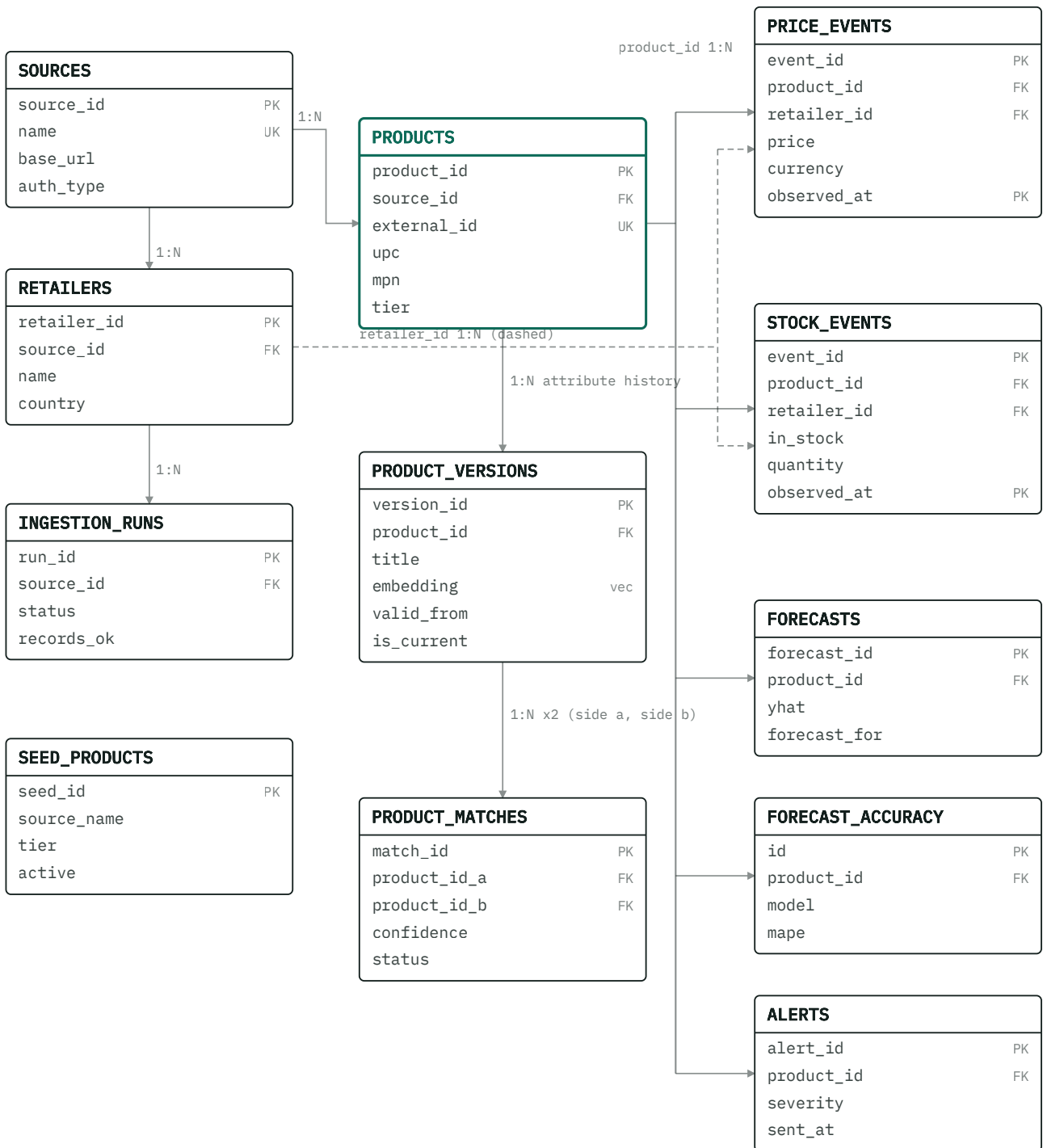
An observation is recorded when the price differs from the preceding one, or when 24 hours have passed since the last record for that product and retailer. The heartbeat matters: without it, an unchanged price is indistinguishable from a source that stopped responding.

Two implementations, one behaviour

Change data capture exists as a readable row-by-row implementation and a set-based SQL implementation whose cost scales with batch size rather than record count. An automated parity test runs both over the same dataset and asserts the resulting rows are identical, covering late arrival, heartbeat boundaries, duplicates and multi-retailer days.

Section 8 — Module 3: Database Design

12 tables in three groups: reference data, append-only event history, and derived output. PostgreSQL 16 with the `pgvector` extension.



PK primary key · FK foreign key · UK unique · vec = 384-dimension vector column · dashed = retailer_id relationship
 price_events and stock_events are range-partitioned by month, so observed_at forms part of their primary key

Figure 4. Entity relationship diagram. `products` is the hub: every table in the right-hand column carries `product_id` as a foreign key, drawn as one bus rather than five crossing lines. The embedding used for product matching lives on `product_versions`, so it versions with the attributes it was derived from.

Constraints enforced by the database

These are constraints in the schema, not conventions in application code, so no application bug can violate them.

GUARANTEE	MECHANISM
Exactly one current version per product	Partial unique index on <code>product_versions(product_id)</code> where <code>is_current</code>
No duplicate observations	Unique index on <code>(idempotency_key, observed_at)</code> , so retries and replays collapse
A match is an unordered pair	Check constraint <code>product_id_a < product_id_b</code> , so one claim cannot be stored twice
Events stay queryable at volume	Monthly range partitions on <code>observed_at</code> , created ahead of time (115 currently in use)

Section 9 — Module 4: Transformation

dbt Core builds the analytical layer in three stages. Staging views clean and type the serving tables. Intermediate views perform the join every mart needs and collapse observations to one price per product, retailer, currency and day. Marts answer specific business questions and are materialised as tables because they are queried repeatedly.

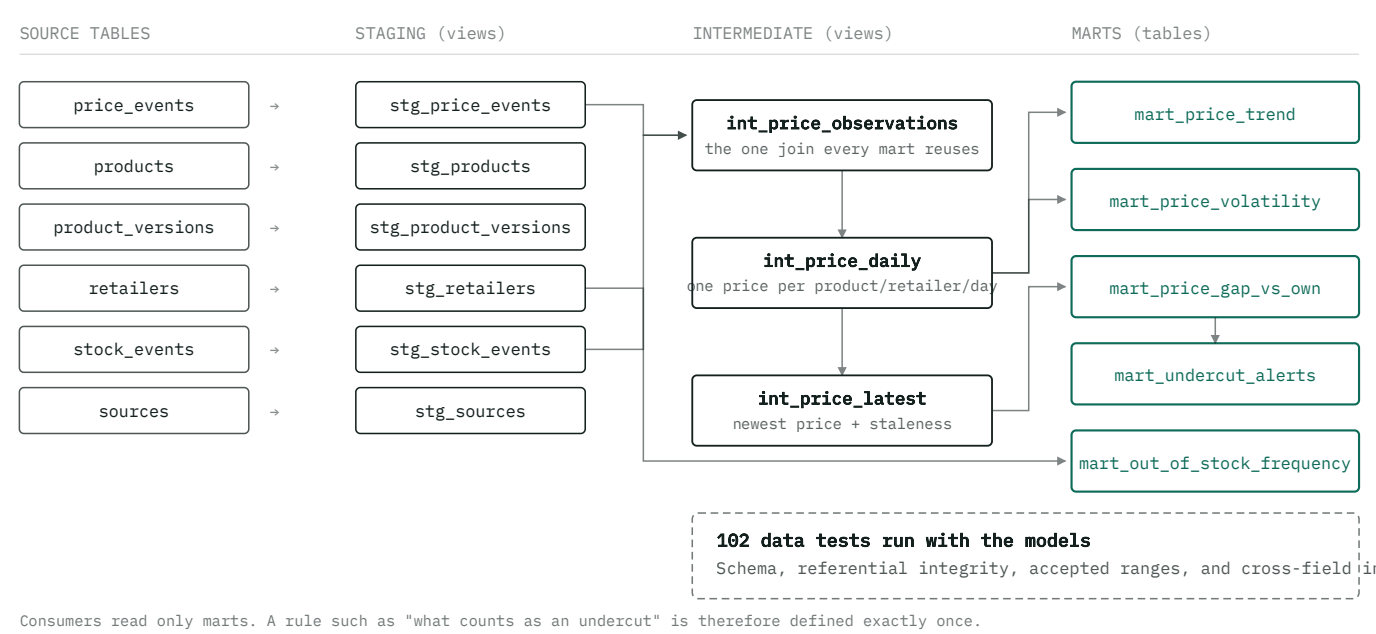


Figure 5. Transformation lineage. Staging cleans, intermediate joins once and defines the daily grain, and marts answer business questions. Tests run with the build, so a failing assertion stops bad numbers reaching the dashboard.

MART	QUESTION IT ANSWERS
<code>mart_price_trend</code>	How has this product's price moved day by day, and what changed yesterday?
<code>mart_price_volatility</code>	Which products swing most in price, and therefore need watching?
<code>mart_price_gap_vs_own</code>	How does each competitor's price compare with ours for the same product?
<code>mart_undercut_alerts</code>	Who is currently cheaper than us, by how much, and how recent is the evidence?
<code>mart_out_of_stock_frequency</code>	How often is a product unavailable at a given retailer?

Currency is part of the grain everywhere. The data spans 24 currencies. Subtracting a price in one currency from a price in another produces a number that looks authoritative and means nothing, and would raise false critical alerts — 12 SEK beside 1.20 EUR reads as a catastrophic undercut when the two are close in value. The gap mart therefore joins on barcode *and* currency, so a cross-currency pair produces no row rather than a wrong one. Coefficient of variation is used as the volatility measure for the same reason: being a ratio, it stays comparable across currencies and price levels.

Section 10 — Module 5: Machine Learning — Forecasting

The platform forecasts short-term price movement per product using statistical time-series models from **Nixtla statsforecast**. Two candidate models compete against a mandatory naive baseline, and the winner is chosen per series on backtested error.

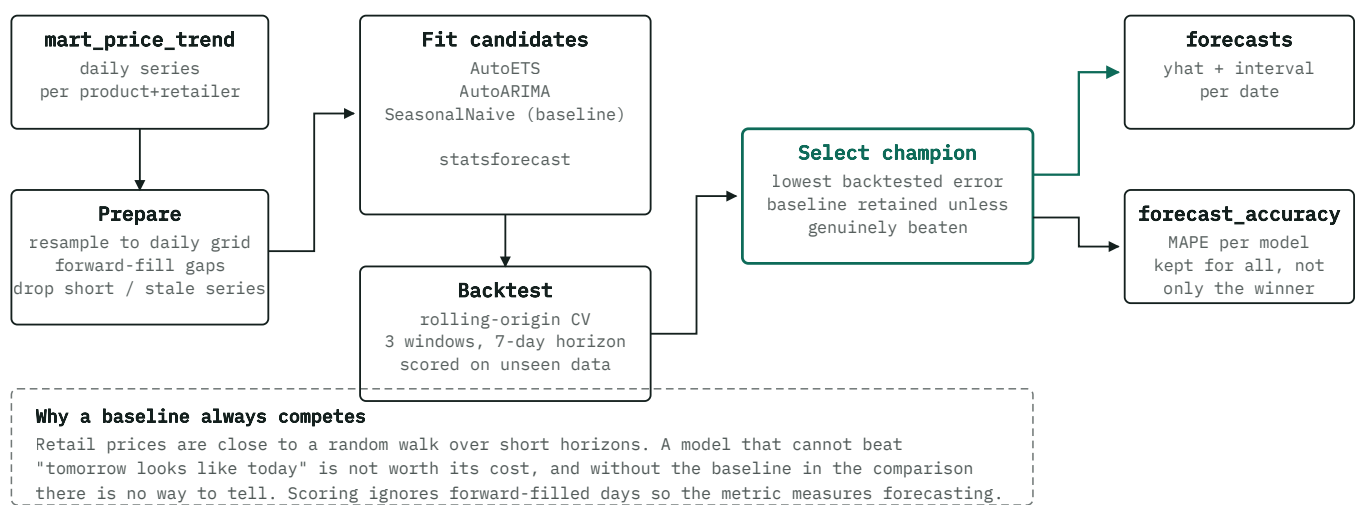


Figure 6. Forecasting module. Accuracy is measured by refitting on successive windows and scoring predictions the model has not seen, which is what makes the reported error meaningful rather than flattering.

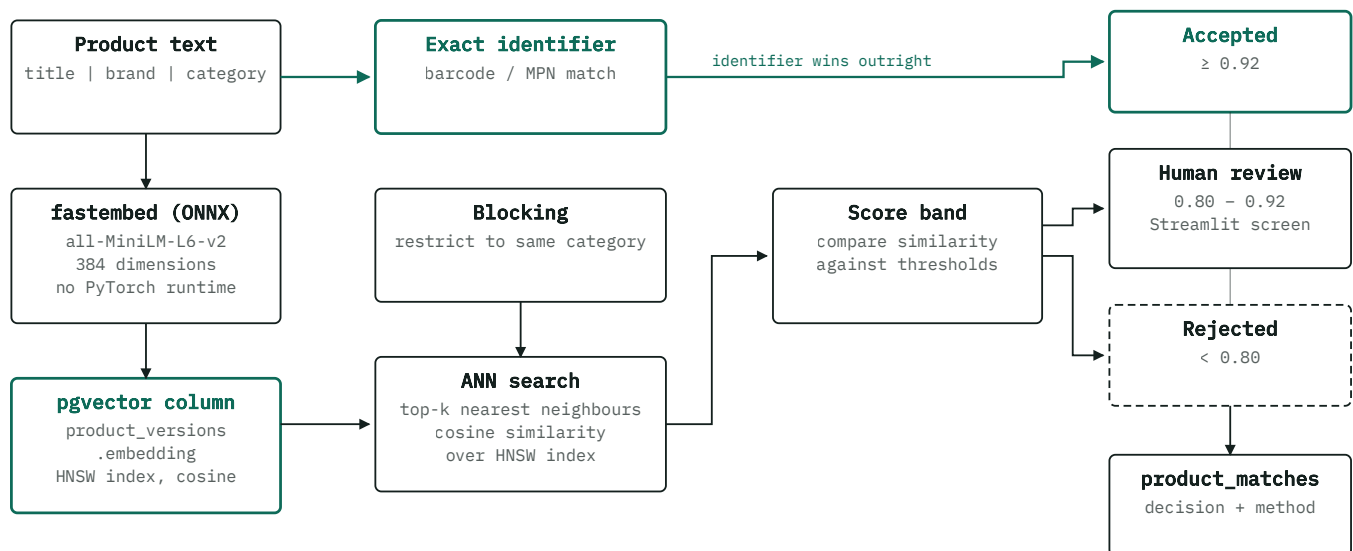
How accuracy is measured

Accuracy is measured by **rolling-origin cross-validation**, not by how well a model fits the data it was trained on. The model is refitted at successive cut-off points and scored only on the days after each cut-off, which it has not seen. The reported metric is MAPE — the average size of the error as a percentage of the actual price.

Prices arrive irregularly, so each series is resampled onto a regular daily grid and carried forward between observations, on the assumption that a shelf price holds until it is next seen. Scoring deliberately ignores those filled days, so the metric measures the forecast rather than the filling. Series that are too short or whose newest observation is older than the staleness threshold are excluded rather than modelled unreliably.

Section 11 — Module 6: Machine Learning — Product Matching

The same physical product appears under different barcodes, titles and languages across retailers. Resolving those into one identity is what makes comparison possible beyond an exact barcode hit. **This is the part of the system that uses vector search.**



An exact barcode is a fact; a similarity score is an opinion. Identifiers therefore bypass the vector path entirely. Reviewer decisions are stored with reviewer and timestamp, forming the labelled set that precision and recall are later measured against.

Figure 7. Product matching. This is where the vector database capability is used: embeddings live in a `pgvector` column on `product_versions`, indexed with HNSW for approximate nearest-neighbour search.

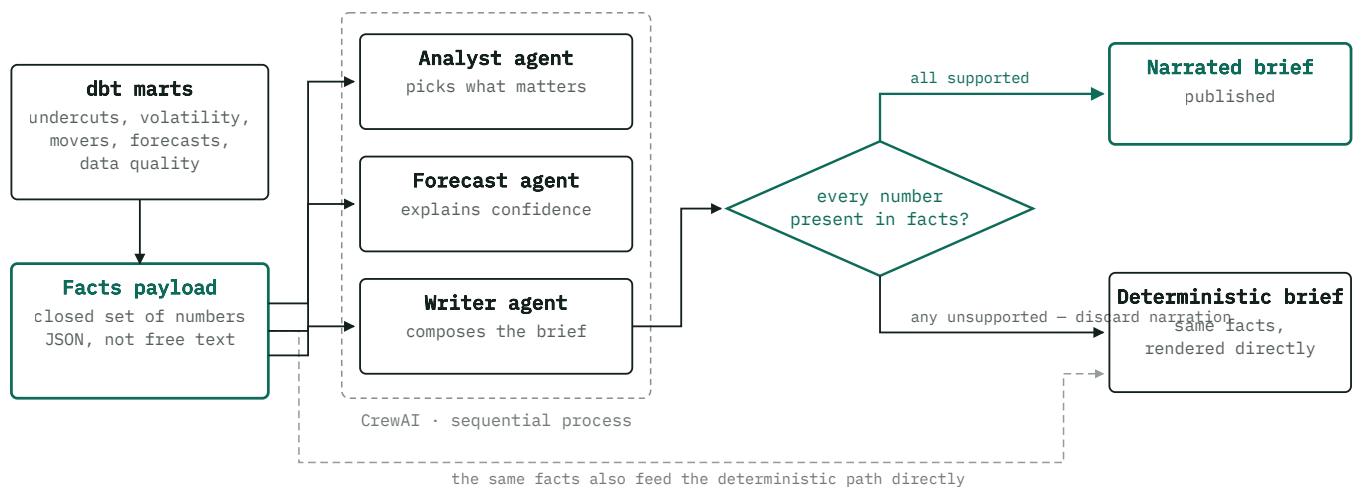
How it works, step by step

1. Each product's `title`, `brand` and `category` are combined into one string and converted by **fastembed** (model `all-MiniLM-L6-v2`) into a **384-dimension vector** — a list of 384 numbers positioning that product in a space where similar meanings sit close together.
2. The vector is stored in the `embedding` column of `product_versions` using the **pgvector** extension, indexed with **HNSW** for fast approximate nearest-neighbour search.
3. An exact barcode or manufacturer part number match is accepted outright and never overridden by a similarity score: an identifier is a fact, a score is an opinion.
4. Otherwise candidates are restricted to the same category first, which cuts the comparison space and suppresses false matches between unrelated products with similar names, and the nearest neighbours are retrieved by cosine similarity.
5. The score is banded: **0.92 and above** accepted automatically, **0.80 to 0.92** sent for human review, **below 0.80** rejected.

Reviewer decisions are stored with reviewer identity and timestamp, which builds a labelled dataset over time — the basis for measuring precision and recall and tuning the thresholds on evidence.

Section 12 — Module 7: Agentic AI — The Weekly Brief

A weekly pricing brief summarises what changed, who is undercutting us, and what the forecasts support. It is produced by a **CrewAI** crew of three role-based agents, and its correctness is enforced mechanically rather than trusted.



The agents never query the database. They receive a fixed payload, which is what makes the check decidable. Worst case is a plainer brief, never one containing a figure the data does not support.

Figure 8. AI module. CrewAI supplies the writing; the guardrail supplies the trust. Because the agents see a closed fact set rather than a database connection, every number they write can be checked mechanically.

The three agents

AGENT	RESPONSIBILITY
Analyst	Selects the few findings that would actually change a pricing decision from the week's movements and undercuts.
Forecast interpreter	Explains what the forecasts and their error rates justify claiming, and says plainly when the evidence is thin.
Writer	Composes the brief in plain language from the analyst's findings and the interpreter's commentary.

Why the agents cannot invent numbers

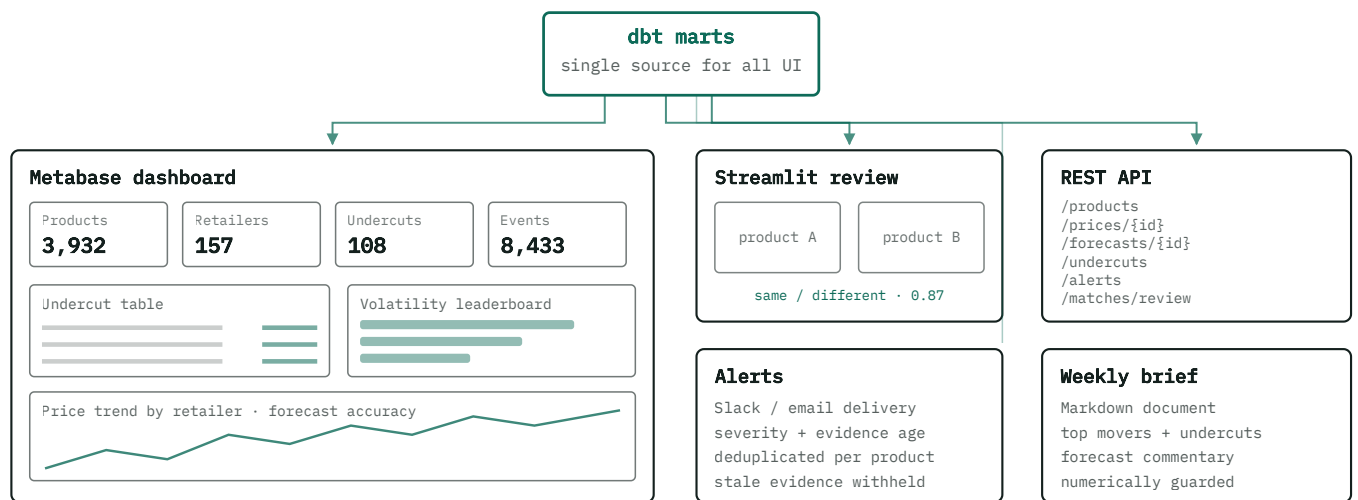
A language model asked to summarise a week will produce plausible figures whether or not they are real, and a pricing brief is precisely the kind of document people act on without re-deriving. Two design decisions remove that risk:

- **The agents never query the database.** A facts builder runs the queries and hands the crew a fixed payload: a closed set of numbers. There is no mechanism by which a figure outside that set can be legitimately produced.
- **Every number in the output is checked against that payload.** If any figure is not present in the facts, the narrated version is discarded and a deterministic rendering of the same facts is published instead.

The worst outcome is therefore a plainer brief, never a brief containing a number the data does not support.

Section 13 — Module 8: Presentation Layer

Five surfaces, each serving a different audience, all reading the same marts.



Five surfaces, one definition. Because each reads the same marts, the undercut count on the dashboard, in the API and in the brief cannot disagree.

Who uses which surface

Category manager → dashboard and weekly brief · Pricing analyst → alerts and API · Data steward → review screen
Engineer → API and pipeline observability

Figure 9. Presentation layer. What each audience sees, and where it comes from. Every surface is downstream of the same tested marts.

ENDPOINT	RETURNS
GET <code>/health</code>	Liveness, database reachability, whether authentication is enabled
GET <code>/products</code>	Tracked catalogue, filterable by category and tier
GET <code>/prices/{product_id}</code>	Daily price history with day-over-day change
GET <code>/forecasts/{product_id}</code>	Latest forecast with prediction intervals
GET <code>/undercuts</code>	Current undercuts, filterable by severity and evidence age
GET <code>/alerts</code>	Raised alerts and their delivery status
GET <code>/matches/review</code>	Candidate product matches awaiting review

Section 14 — Field Legend (Data Dictionary)

Column names shown on the dashboard and in the API are defined below in plain language, grouped by where they appear. Names highlighted in **green** are the ones whose dashboard label differs from the underlying column.

Price and observation fields

FIELD	APPEARS IN	WHAT IT MEANS
observed_at	price_events, marts	When the price was seen in the shop — the source's own timestamp, not ours.
ingested_at	price_events	When our system stored it. The gap between the two is how late the data arrived.
currency	everywhere	ISO code such as EUR or SEK. Part of the grain: prices are never compared across currencies.
close_price	mart_price_trend	The last price recorded for that product, retailer and day. 'Close' as in end-of-day.
change_pct	mart_price_trend	Percentage change from the previous recorded day. Negative means the price fell.
idempotency_key	price_events	A fingerprint of source, product, retailer and time. Stops a retry recording the same observation twice.

Competitor comparison and alerting fields

FIELD	APPEARS IN	WHAT IT MEANS
our_price	gap and undercut marts	The price in our own catalogue for the matched product.
competitor_price	gap and undercut marts	The most recent price seen at that competitor.
gap_abs	mart_price_gap_vs_own	Their price minus ours, in currency. Negative = they are cheaper.
gap_pct	gap and undercut marts	The same difference as a percentage of our price. -28.70 means they are 28.7% below us.
is_undercut	mart_price_gap_vs_own	True when the competitor is cheaper than us.
severity	mart_undercut_alerts	How large the undercut is: critical at 20% or more below us, high at 10–20%, medium under 10%.
days_stale (shown as 'Age')	undercut mart, alerts	How many days old the evidence is. 0 = seen today. Large values mean the price may have changed since.
confidence	mart_undercut_alerts	A plain label for that age: fresh (0–1 days), recent (2–7 days), stale (over 7 days). Alerts on stale evidence are recorded but not sent.

Volatility fields

FIELD	APPEARS IN	WHAT IT MEANS
mean_price	mart_price_volatility	Average price across all recorded days for that product at that retailer.
min_price / max_price	mart_price_volatility	Cheapest and dearest recorded values, giving the observed range.
stddev_price	mart_price_volatility	Standard deviation — how far prices typically sit from the average, in currency.
coefficient_of_variation (shown as 'CV')	mart_price_volatility	Standard deviation divided by the mean. Being a ratio it has no unit, so a €0.35 baguette and a €3.57 spread can be ranked on the same scale. 0.20 means prices typically swing about 20% around the average.
volatility_band (shown as 'Band')	mart_price_volatility	A readable bucket for that ratio: high at 0.20 and above, medium 0.05–0.20, low below 0.05.
observation_count (shown as 'Points')	mart_price_volatility	How many separate days went into the statistics. More points means a more trustworthy figure.

Modelling, forecasting and matching fields

FIELD	APPEARS IN	WHAT IT MEANS
tier	products, seed_products	How often we re-check this product: 1 = every 6 hours, 2 = daily, 3 = weekly. Used to spend limited API quota on the products that move most.
is_current	product_versions	Marks the one row describing the product as it is now . Older rows stay for history.
valid_from / valid_to	product_versions	The window during which that description was accurate. <code>valid_to</code> is empty on the current row.
embedding	product_versions	The 384-number vector representing the product's text. Used to find similar products; not human-readable.
yhat	forecasts	The predicted price. 'y-hat' is the standard statistical notation for a predicted value.
yhat_lower / yhat_upper	forecasts	The prediction interval: the range the price is expected to fall within, at 80% confidence.
horizon_days	forecasts	How far ahead the prediction reaches — here, 7 days.
mape	forecast_accuracy	Mean Absolute Percentage Error: average size of the model's mistakes, as a percentage. 3.13 means predictions were off by about 3% on average. Lower is better.
confidence (matching)	product_matches	Cosine similarity between two product vectors, from 0 to 1. 1.00 = identical text; 0.92 and above is treated as the same product.

FIELD	APPEARS IN	WHAT IT MEANS
method	product_matches	How the pair was matched: exact_upc (same barcode), embedding_auto (vector similarity, accepted), embedding_review (awaiting a person), human (decided by a reviewer).
status	product_matches	approved , pending or rejected . Only approved pairs are treated as the same product.

Worked example

Reading one row of the undercut table end to end:

Product	Riz aux Champignons de Paris	
Retailer	U Express	
Ours	2.23 EUR	← our_price, from our catalogue
Theirs	1.59 EUR	← competitor_price, most recent observation
Gap	-28.70%	← gap_pct: they are 28.7% cheaper than us
Severity	critical	← because the gap is 20% or more
Age	6 days	← days_stale: the observation is six days old
Evidence	recent	← confidence label for an age of 2 to 7 days

Read as a sentence: *at U Express this product is 28.7% cheaper than our price, which is a critical gap, based on evidence gathered six days ago.*

Section 15 — AWS Deployment Plan

The platform is designed for a single-instance deployment in **ap-south-1 (Mumbai)**, defined end to end in Terraform. The design goal is a credible production topology at portfolio cost: the database is unreachable from the internet, secrets never appear in code or images, and the monthly bill is bounded and alarmed.

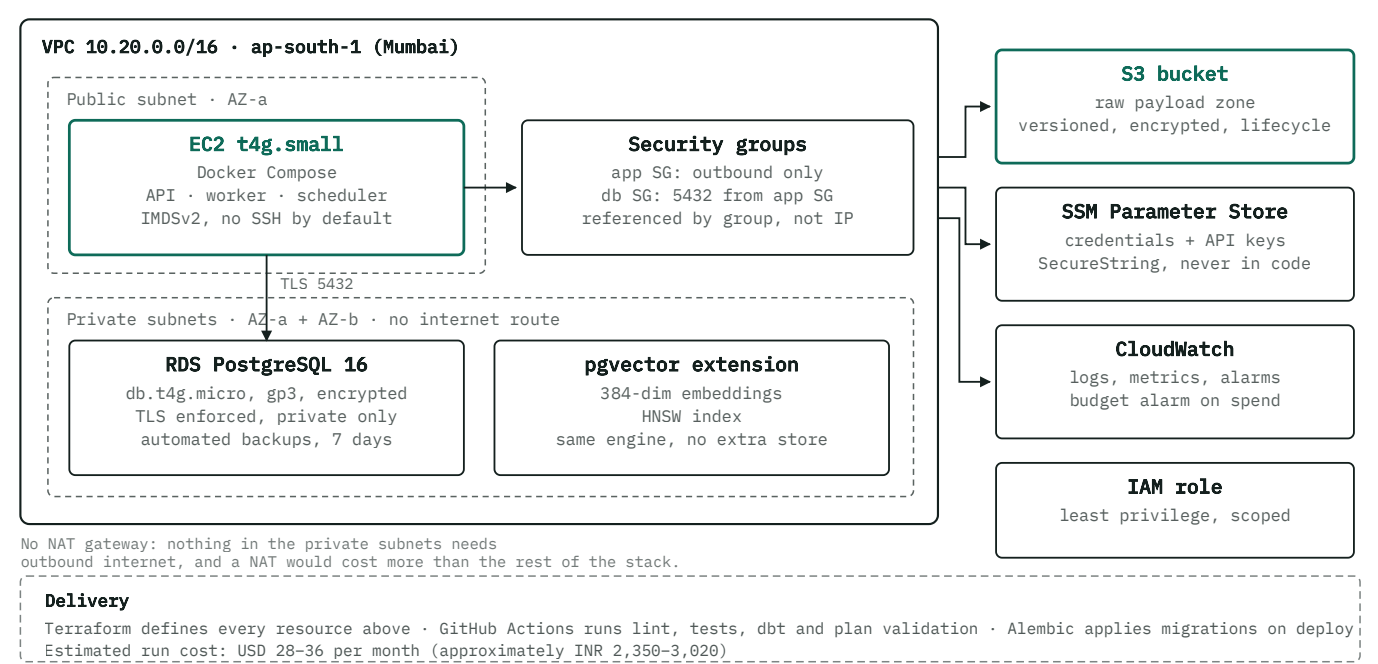


Figure 10. Target AWS deployment. The database is unreachable from the internet by construction: it sits in subnets with no internet route and accepts traffic only from the application's security group.

Services and their role

SERVICE	CONFIGURATION	ROLE IN THIS PLATFORM	USD/MO
Amazon EC2	t4g.small, public subnet	Runs the API, Celery worker and scheduler as containers. Graviton (ARM) for roughly 20% lower cost than the x86 equivalent.	~\$12-15
Amazon RDS for PostgreSQL	db.t4g.micro, 20 GB gp3, Single-AZ	The serving database, with pgvector enabled. Private subnets, TLS enforced, 7-day automated backups.	~\$14-16
Amazon S3	One bucket, versioned	Raw payload zone. Lifecycle rules move objects to infrequent access at 90 days and archive at 365.	~\$1-2
AWS Systems Manager Parameter Store	SecureString parameters	Database URL and third-party API keys. Chosen over Secrets Manager because SecureString is free at this scale.	included
Amazon CloudWatch	Logs, metrics, alarms	Structured application logs, CPU and storage alarms, and a monthly budget alarm on spend.	~\$1-3
AWS IAM	One instance role	Least privilege: read this project's parameters, read and write this bucket's objects, write its own log group. No wildcards.	free

SERVICE	CONFIGURATION	ROLE IN THIS PLATFORM	USD/MO
Amazon VPC	1 public + 2 private subnets	Network isolation. The database subnets have no internet route, so there is no NAT gateway to pay for.	free

Cost summary

SCENARIO	USD / MONTH	INR / MONTH
Single EC2 + RDS + S3 (this design)	28–36	2,350–3,020
Container-orchestrated alternative (Fargate + load balancer + NAT)	~108	~9,070
Development on managed free tiers	0	0

The container-orchestrated alternative is rejected on cost, not capability: the compute itself is inexpensive, but a load balancer, NAT gateway and log retention add roughly USD 90 per month per environment — several times the rest of the stack. The single-instance design is the appropriate scale for this workload, and the migration path to containers is straightforward should throughput demand it.

Deployment mechanics

- **Infrastructure as code.** Terraform defines the VPC, subnets, security groups, EC2 instance, RDS instance and parameter group, S3 bucket with lifecycle rules, SSM parameters, IAM role and policies, CloudWatch alarms and the budget alarm.
- **Configuration at boot.** The instance fetches its secrets from Parameter Store on start-up into a root-only environment file. Nothing sensitive is baked into the image or committed to the repository.
- **Continuous integration.** GitHub Actions runs linting, the Python test suite, the dbt build and Terraform validation on every push, without requiring cloud credentials.
- **Migrations.** Alembic applies schema changes on deploy, including creation of the next months' table partitions.
- **Cost control.** A budget alarm notifies on both forecast and actual spend, so an unexpected charge surfaces before the month ends rather than after.

Section 16 — Security and Access Control

CONTROL	IMPLEMENTATION
Database isolation	RDS has no public address and sits in subnets with no internet route. Its security group accepts PostgreSQL traffic only from the application security group, referenced by group rather than by IP so it survives instance replacement.
Encryption	Storage encrypted at rest on both RDS and S3; TLS enforced on database connections by parameter group.
Secret handling	Credentials and API keys held as SecureString parameters, fetched at boot, never written to the repository or image.
Least privilege	One instance role scoped to this project's parameter path, this bucket's objects and its own log group. No wildcard resources.

CONTROL	IMPLEMENTATION
Instance access	SSH closed by default; administrative access via Session Manager, so no port is exposed and no key material is distributed. IMDSv2 required.
API authentication	API-key header with constant-time comparison; the health endpoint reports whether authentication is active so an open deployment is visible.
Data protection	No personal data is collected. The platform stores public product prices, retailer names and product attributes only.

Section 17 — Testing and Data Quality

LAYER	TESTS	WHAT IS VERIFIED
Python unit	~130	Normalizers, CDC rules, forecast metrics, alert logic, the numeric guard, authentication
Python integration	~20	Rate limiting against a real Redis; CDC parity between both implementations against a real database
dbt data tests	102	Schema, referential integrity, accepted ranges and cross-field invariants
Infrastructure	—	Terraform formatting and validation in continuous integration

The data tests assert relationships *between* fields rather than only per-column constraints. The computed gap must agree with its own inputs; the undercut flag must agree with the gap; a maximum must not fall below its minimum; time must move forward within a series. A simple not-null test would catch none of these, and each guards against a refactor silently inverting a rule.

Section 18 — Results

Platform scale, measured with the pipeline idle and marts freshly built

MEASURE	VALUE
Price observations recorded	8,433
Observations within the last 30 days	3,736
Products tracked	3,932
Retailers observed	157
Currencies handled	24
Monthly partitions in use	115
Undercuts detected	108
Undercuts on evidence 7 days old or less	62
Products embedded for matching	3,848
Product pairs matched	2,557
Automated tests passing	252

Largest undercuts on current evidence (EUR)

PRODUCT	RETAILER	OURS	THEIRS	GAP	AGE
Riz aux Champignons de Paris	U Express	2.23	1.59	-28.70%	6
Sriracha Hot Chilli Sauce	Super U	4.33	3.14	-27.48%	4
Tabasco Sauce Epicee Rouge	U Express	3.86	2.86	-25.91%	5
Sauce Quick Supreme Spicy	U Express	2.94	2.18	-25.85%	5
Amora Sauce Samourai 255g	E.Leclerc Express	2.16	1.65	-23.61%	6

Most volatile products by coefficient of variation

PRODUCT	RETAILER	MEAN	CV	BAND	POINTS
Petites Madeleines	E. Leclerc	2.43	0.455	high	5
Lentilles vertes	Centre Commercial E.Leclerc	1.21	0.375	high	7
PESTO ROSSO	E.Leclerc	2.46	0.370	high	5
PESTO ROSSO	Carrefour Market	2.34	0.352	high	6
Mais doux en grains	Centre Commercial E.Leclerc	0.50	0.345	high	8

Forecast accuracy, backtested on unseen windows

MODEL	SERIES	AVG MAPE	ASSESSMENT
AutoARIMA	9	3.13%	Within the 12% target
AutoETS	9	3.13%	Within the 12% target
SeasonalNaive (baseline)	9	3.13%	Reference point for comparison

Engineering results

OPERATION	RESULT
Replay of one stored day (947 payloads, 7,922 records)	41 s
Full warehouse rebuild from the raw zone, no API calls	30 s
Embedding generation, 1,251 products	35 s
Dashboard card query latency	141–486 ms

The rebuild figure is the one worth noting. Because every raw payload is stored before parsing, the entire warehouse is reconstructible from object storage without contacting any source API — verified in practice, not only in design.

Section 19 — Roadmap

1. **Connect a keyed retail source, starting with Best Buy.** It carries stock levels, which activates availability analytics, and its refresh cadence supplies denser daily series for forecasting. Highest value per unit of integration effort.

2. **Grow the reviewed match set** to a few hundred labelled pairs, then measure precision and recall and tune the similarity thresholds on that evidence.
3. **Add dated foreign-exchange rates** to enable cross-currency comparison. The change is additive: a rates table, a converted column in the intermediate layer, and currency-agnostic marts alongside the existing scoped ones.
4. **Introduce a challenger forecasting model** (gradient-boosted trees with calendar and promotion features) for products with enough history to support it.
5. **Adopt an orchestrator** such as Dagster or Airflow when backfill lineage across interdependent jobs becomes the operational constraint.

Appendix — Running the Platform

```
docker compose up -d                # PostgreSQL + Redis
alembic upgrade head                 # schema, partitions, extensions
python -m app.cli ingest openprices --limit 600  # broad discovery sweep
python -m app.cli ingest openprices --seeds      # deep per-SKU history
python -m app.cli seed openprices --tier 1      # choose tracked SKUs
make dbt-build          # transformation models + 102 data tests
make forecast           # backtest and write forecasts
make match              # embed products and generate match candidates
make alerts             # evaluate undercuts and deliver alerts
make brief              # weekly pricing brief
make review             # match review UI          make metabase  # BI dashboard
make api                # REST API
```

Figures in this report were read from the running platform on 1 September 2026 with the pipeline idle and the marts freshly rebuilt.